Literature Review: Software Metrics in Continuous Integration and Continuous Delivery Pipelines

CSE 7319 - Software Metrics and Quality

December 2, 2025

1 Introduction

This literature review examines recent research on internal and external software metrics applied to Continuous Integration and Continuous Delivery (CI/CD) pipelines.

2 Internal Metrics

Internal metrics depend on program artifacts and can be measured through static or dynamic analysis. As independent variables in the metrics relationship model, they serve as leading indicators and control mechanisms for achieving external quality goals. Classic categories include size metrics (LOC, function points), complexity metrics (cyclomatic complexity, Halstead measures, data/control flow), and structure metrics (coupling, cohesion). The papers reviewed here focus on how internal metrics are collected and applied within CI/CD pipelines, particularly static analysis quality checks and test coverage measurements.

2.1 Static Code Analysis Metrics

How Open Source Projects Use Static Code Analysis Tools in Continuous Integration Pipelines [?]

2.1.1 Summary

This empirical study investigates how open-source projects integrate static code analysis tools into their continuous integration workflows. The researchers analyzed real-world CI configurations to understand what types of code quality issues cause builds to fail versus generate warnings, and how developers respond to these warnings or failures. The study examined projects using the following tools: Checkstyle, PMD, and FindBugs integrated into CI systems.

A key finding was 97% of build warnings and 6% of build failures were triggered by style issues rather than bug detection. The researchers also found that when static analysis generates warnings or failures, developers typically address the issues directly by fixing the code rather than suppressing warnings or adjusting tool configurations.

2.1.2 Analysis

This paper demonstrates the practical application of static analysis of internal metrics within CI/CD pipelines. The finding that coding style violations constitute the majority of build failures indicates teams using these tools are attempting to enforce code maintainability. This illustrates a predictive relationship, by controlling internal metrics such as code style and structural consistency, teams aim to improve external metrics of maintainability.

The paper validates that adding a quality gate within the build pipeline effectively enforces internal code style requirements. The developers' tendency to fix issues rather than suppress warnings demonstrates that when internal metrics are properly integrated into workflows, they influence behavior and drive quality improvements. From a GQM perspective, these teams have defined maintainability goals, posed questions about code consistency,

and selected metrics such as style violations and complexity checks.

2.2 Test Coverage Metrics

Open Code Coverage Framework: A Consistent and Flexible Framework for Measuring Test Coverage [?]

2.2.1 Summary

This paper addresses a fundamental challenge in measuring test coverage across different programming languages and coverage criteria: the high cost and inconsistency of coverage measurement tools. The researchers observed that while test coverage is a critical quality indicator, each language typically requires separate tools with inconsistent reporting formats and capabilities.

They developed the Open Code Coverage Framework (OCCF), which abstracts common concepts across programming languages using abstract syntax trees (ASTs). By identifying structural commonalities at the AST level, the framework can support multiple languages with a single common solution. The authors demonstrated that their approach reduced implementation requirements by approximately 90% and cut development time for new coverage criteria by 80% or more. The framework supports various coverage types including statement, decision, branch, and path coverage while remaining extensible for custom criteria.

2.2.2 Analysis

This paper addresses a fundamental challenge in measurement theory: ensuring consistent and properly defined metrics across different contexts. Test coverage is an internal metric, and serves as a leading indicator for external quality attributes like reliability and defect density. The framework's abstraction approach of using ASTs to identify structural commonalities enables uniform measurement of statement, branch, and path coverage across

languages. This consistency is essential for establishing valid predictive models: if coverage is measured differently across components, correlations with external quality metrics become unreliable.

2.3 CI/CD Metrics Visualization

A Roadmap for Using Continuous Integration Environments [?]

2.3.1 Summary

This study investigates how development teams can leverage CI environments to generate and visualize quality metrics. Through interviews with 22 participants across five industrial projects, the researchers identified that 86% of participants found CI-generated data useful for quality attribute evaluation, and 73% valued centralized dashboards for visualizing this data. The study cataloged the following metrics that can be captured throughout a project's lifecycle:

Category	Metrics
Version Control	No. of Commits
Source Code	Lines of Code, No. of Source Files
Continuous Integration	Time Stamps, No. of Builds
Static Code	Lines of Code, cyclomatic Complexity, Code Smells
Artifacts	No. of Release Versions, Release Version Tags
Test	Mean Recovery Time
Performance	No. of Service Requests, CPU/Memory Usage, No. of Transactions, No. of Vulnerabilities
Issue Tracking	No. of Tickets

Performance metrics showed the highest adoption at 82% among participants. The paper provides concrete examples of metrics collection pipelines using Prometheus for data gathering and Grafana for visualization. The researchers also propose a three-level maturity model (elementary, intermediate, advanced) for CI environment evolution, finding that 60% of studied projects operated at intermediate maturity levels. Challenges identified include dashboard information overload and insufficient understanding of metrics meanings.

2.3.2 Analysis

This paper directly addresses the metrics usage workflow: after measurement activities collect data, analysis and visualization are essential to make results actionable. The cataloging of 14 base and 10 derived metrics demonstrates the variety of internal and external metrics available in modern CI/CD systems. Performance metrics showing the highest adoption reflects their direct connection to external user experience, while the centralized dashboard preference indicates that integrated visualization of multiple metrics is more valuable than isolated measurements. The identified challenges—information overload and insufficient understanding—highlight a pitfall, where the overcollection of metrics could dilute their importance. This suggests the value for an "Experience Factory" approach where organizations identify which metrics matter and how to interpret them.

3 External Metrics

External metrics reflect quality from the customer and user perspective, measuring the "-ilities" (reliability, usability, performance, security, maintainability) that determine system value. As dependent variables in the metrics relationship model, they must be predicted and managed through monitoring of internal metrics.

3.1 DevOps Research and Assessment (DORA) Metrics

A Framework for Automating the Measurement of DORA Metrics [?]

3.1.1 Summary

The DORA metrics of Deployment Frequency, Lead Time for Change, Mean Time to Recover, and Change Failure Rate, have become industry-standard indicators of software delivery performance. This paper presents a framework that automatically extracts and calculates DORA metrics from project repositories without requiring specific CI/CD tools or cloud platforms.

The researchers validated their framework by analyzing 304 popular open-source projects on GitHub, generating time-series data that revealed meaningful insights into project performance trends. A key contribution is the project-agnostic design: the framework can operate across diverse development lifecycles and deployment approaches. The analysis showed that significant changes in DORA metrics correlated with major historical events in projects, validating the metrics' ability to detect meaningful shifts in development performance.

Percentile	Deployment Frequency	Lead Time for Changes	Mean Time to Recover	Change Failure Rate
Top 25%	116.2	5.1	26.5	19.8%
Middle 50%	26.2	26.8	97.9	59.4%
Bottom 25%	6.6	140.6	370.0	90.6%

- Values are calculated as the mean of all projects within the percentile.
- Deployment Frequency represents median deployments per year.
- Lead time for changes represents median days between commit and deployment.
- Mean Time to recover represents median days between failure and recovery.

3.1.2 Analysis

The DORA metrics represent external quality attributes in the DevOps context, aligned with the Putnam/Myers framework's external metrics of time, quality, and productivity:

- Deployment frequency and lead time reflect productivity and schedule performance
- Change failure rate and time to restore service measure reliability.

The framework's project-agnostic design is significant because it enables consistent measurement across diverse environments. This paper demonstrates that external metrics can be extracted automatically from development artifacts such as commits, deployments, and incidents. The findings of this study from the sampled github project provides an important baseline to compare the relative reliability, productivity, and schedule performance of any new software project.

3.2 Requirements Traceability

Continuous Integration and Delivery Traceability in Industry: Needs and Practices [?]

3.2.1 Summary

The paper documents the Eiffel framework, developed within Ericsson to provide real-time traceability throughout CI/CD pipelines. The Eiffel approach defines a common protocol to model test events as a directed acyclic graph, enabling traceability to be directly embedded into the continuous integration infrastructure. This methodology allows automatic capture of relationships between requirements, commits, builds, tests, and deployments as they occur. This enables immediate visibility into which requirements are affected by changes and what testing has been performed.

The payload of each Eiffel message is a JSON document containing links to other uniquely identified test events and information regarding the test event itself, such as test information,

and test result (Pass/Fail/Warn). Each message is published to a common database, enabling aggregation, visualization, and analysis of test events.

3.2.2 Analysis

Requirements traceability connects external requirements like customer needs and specified functionality to internal artifacts such as commits, builds, or tests. The Eiffel framework embeds automatic measurement directly into development workflows, addressing the classic challenge that manual metrics collection is impractical in fast-paced development. This automation enables real-time tracking rather than periodic snapshots, making traceability a useful leading indicator: when a requirement changes, teams immediately know which code and tests are affected.

From a measurement theory perspective, this automated approach produces more reliable and centralized data than manual maintenance because it captures actual relationships, like commits linked to requirements, rather than documented intentions. The integration of traceability with CI/CD metrics enables teams to track if specific requirements are fully implemented and tested, and reduces the labor overhead of collecting and aggregating test results.

3.3 Security Metrics and Temporal Tracking

Security Regression Visualization in CI/CD Using Trivy and GitHub Actions [?]

3.3.1 Summary

This paper addresses a critical gap in DevSecOps practices: while vulnerability scanners like Trivy are widely integrated into CI/CD pipelines, they typically operate in a point-in-time manner, generating alerts without retaining historical context. The researchers propose a lightweight security regression tracking system that persists, visualizes, and analyzes vulnerability trends across multiple builds.

The implementation combines three components: Trivy for container image scanning, GitHub Actions for automation, and Chart.js for visualization. At each build, the pipeline scans the Docker image, extracts vulnerability counts by severity level (Critical, High, Medium, Low), and appends results to a time-series CSV file. A static web dashboard hosted on GitHub Pages renders trend visualizations, enabling teams to assess whether their security posture is improving or degrading over time.

3.3.2 Analysis

The approach demonstrates a time-based metrics relationship model by tracking how internal changes such as dependency updates, code modifications, and base image selection influence external security metrics. The automated logging mechanism ensures consistent measurement across builds.

4 Conclusion

The reviewed papers demonstrate how modern CI/CD environments enable comprehensive software quality measurement through three interconnected capabilities: automated metrics collection, event-based aggregation, and centralized visualization. Together, these capabilities transform software metrics from manual, periodic measurements into continuous, actionable intelligence.

Metrics Collection: The papers identify diverse metrics collected automatically throughout the CI/CD pipeline. Internal metrics include code complexity and style violations (Checkstyle, PMD, FindBugs), test coverage at statement, branch, and path levels (OCCF), and lines of code measurements. External metrics encompass DORA's delivery performance indicators (Deployment Frequency, Lead Time for Changes, Mean Time to Recover, Change Failure Rate), security vulnerability counts by severity level (Trivy scans), and requirements traceability status. The research demonstrates that these metrics can be extracted automat-

ically from version control systems, static analysis tools, test frameworks, container scanners, and deployment systems without manual intervention. This automated collection ensures measurement consistency across builds and enables continuous monitoring rather than periodic snapshots.

Metrics Aggregation: Once collected, metrics must be aggregated and correlated to provide meaningful insights. The Eiffel framework exemplifies this capability by defining a common protocol for embedding traceability throughout CI/CD pipelines. Using JSON-based event messages organized as a directed acyclic graph, Eiffel enables automatic capture of relationships between requirements, commits, builds, tests, and deployments. Each event links to related events through unique identifiers, creating a comprehensive data model that connects internal code changes to external quality outcomes. This aggregation layer transforms isolated measurements into correlated time-series data, enabling teams to understand how code modifications propagate through the pipeline and affect multiple quality dimensions simultaneously.

Metrics Visualization: The final capability—visualization through dashboards—makes aggregated metrics actionable for development teams. The papers document successful implementations using Prometheus for data gathering combined with Grafana for interactive dashboards, as well as lightweight alternatives like Chart.js with GitHub Pages for security trend visualization. These dashboards centralize multiple metric types, providing unified views of code quality, test adequacy, delivery performance, and security posture. The research shows that 86% of practitioners find CI-generated data useful for quality evaluation, and 73% value centralized dashboards. However, challenges remain: information overload and insufficient understanding of metrics meanings can reduce effectiveness, suggesting the need for carefully designed visualizations that highlight actionable insights rather than raw data.

By integrating automated collection, protocol-based aggregation, and dashboard visualization, modern CI/CD environments realize the vision of continuous quality measurement.

This three-layer architecture enables teams to track both internal leading indicators and external outcome measures, establish correlations between them, and respond to quality trends before they impact users—transforming metrics from descriptive statistics into active controls that guide continuous improvement.